

Perceptron Model

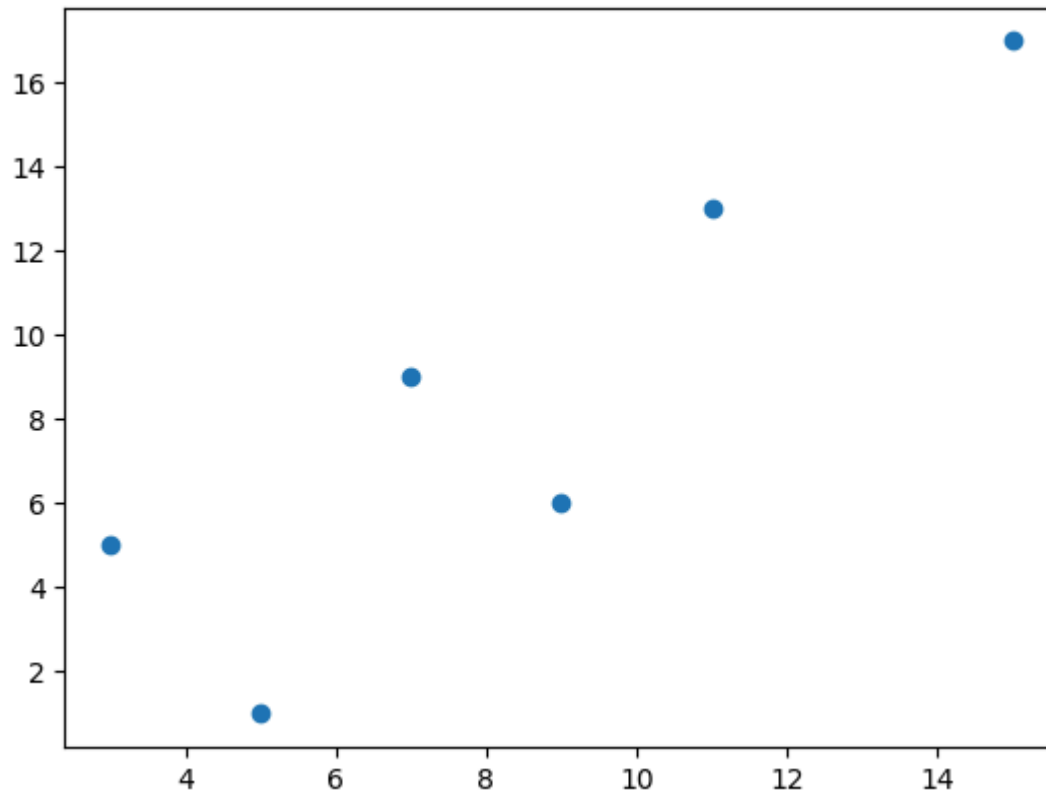
```
In [ ]: import keras
import tensorflow
```

```
In [ ]: from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam

import numpy as np
import matplotlib.pyplot as plt
```

```
In [ ]: x = np.array([3,7,5,9,11,15])
y = np.array([5,9,1,6,13,17])
```

```
In [ ]: plt.scatter(x,y)
plt.show()
```



```
In [ ]: #Perceptron Model
model = Sequential()
#input & hidden layer
model.add(Dense(1,input_dim=1,activation='linear'))
#output layer
model.add(Dense(1,activation='linear'))
model.compile(Adam(learning_rate=0.01),loss='mse')
```

```
In [ ]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 1)	2
dense_3 (Dense)	(None, 1)	2



Total params: 4 (16.00 B)

Trainable params: 4 (16.00 B)

Non-trainable params: 0 (0.00 B)

```
In [ ]: model.fit(x,y,epochs=20,verbose=1)
```

```
Epoch 1/20
1/1 ————— 0s 43ms/step - loss: 378.3142
Epoch 2/20
1/1 ————— 0s 59ms/step - loss: 371.9059
Epoch 3/20
1/1 ————— 0s 48ms/step - loss: 365.6587
Epoch 4/20
1/1 ————— 0s 66ms/step - loss: 359.5685
Epoch 5/20
1/1 ————— 0s 46ms/step - loss: 353.6310
Epoch 6/20
1/1 ————— 0s 48ms/step - loss: 347.8423
Epoch 7/20
1/1 ————— 0s 49ms/step - loss: 342.1982
Epoch 8/20
1/1 ————— 0s 47ms/step - loss: 336.6949
Epoch 9/20
1/1 ————— 0s 59ms/step - loss: 331.3285
Epoch 10/20
1/1 ————— 0s 62ms/step - loss: 326.0952
Epoch 11/20
1/1 ————— 0s 58ms/step - loss: 320.9914
Epoch 12/20
1/1 ————— 0s 47ms/step - loss: 316.0133
Epoch 13/20
1/1 ————— 0s 51ms/step - loss: 311.1577
Epoch 14/20
1/1 ————— 0s 46ms/step - loss: 306.4209
Epoch 15/20
1/1 ————— 0s 61ms/step - loss: 301.7998
Epoch 16/20
1/1 ————— 0s 47ms/step - loss: 297.2910
Epoch 17/20
1/1 ————— 0s 46ms/step - loss: 292.8913
Epoch 18/20
1/1 ————— 0s 47ms/step - loss: 288.5978
Epoch 19/20
1/1 ————— 0s 60ms/step - loss: 284.4073
Epoch 20/20
1/1 ————— 0s 60ms/step - loss: 280.3170
```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7a0b43721e90>
```

```
In [ ]: model.predict(x)
```

```
1/1 ————— 0s 62ms/step
```

```
Out[ ]: array([[ -1.4451506],
              [ -4.7782793],
              [ -3.1117148],
              [ -6.4448423],
              [ -8.111406 ],
              [-11.444535 ]], dtype=float32)
```

```
In [ ]:
```

```
In [ ]:
```

Multi-Layered Perceptron

For Continuous Output

```
In [ ]: #Importing Packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: #Reading and Studying the dataset
adv = pd.read_csv(r'/content/advertising.csv')
adv
```

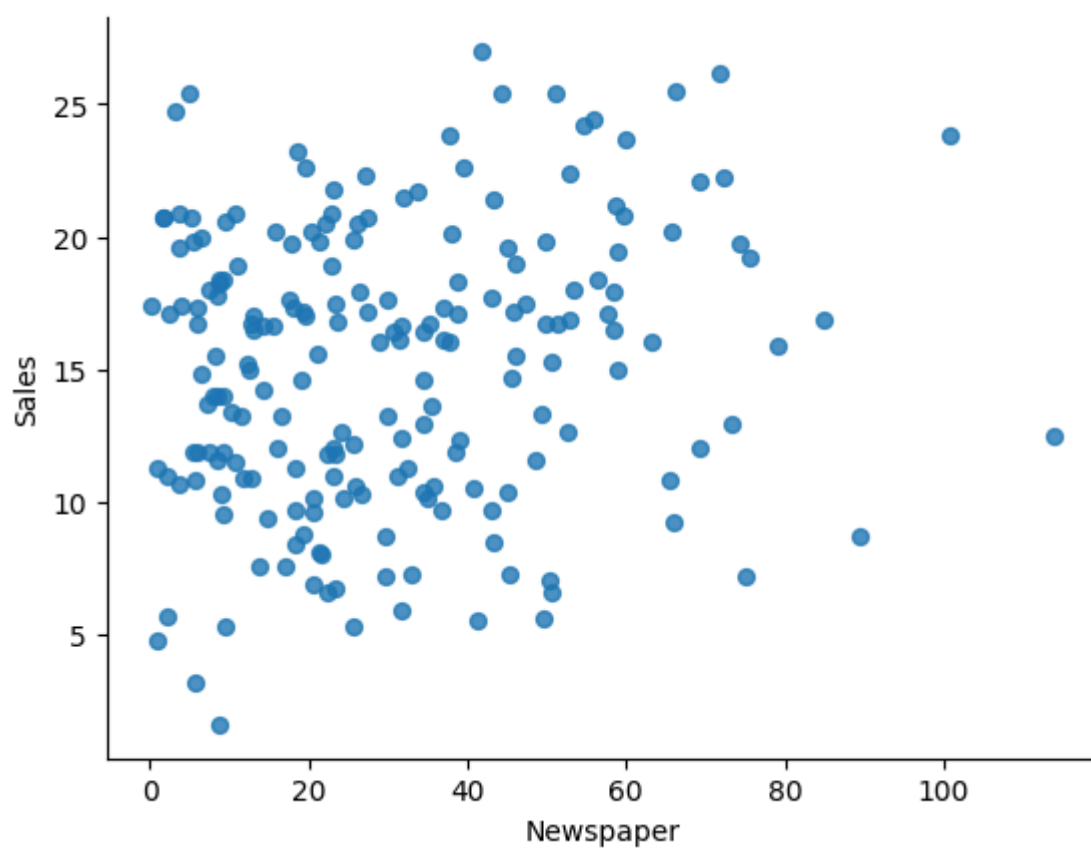
```
Out[ ]:
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9
...	...	...	...	...
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	14.0
197	177.0	9.3	6.4	14.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	18.4

200 rows × 4 columns

```
In [ ]: # @title Newspaper vs Sales

from matplotlib import pyplot as plt
adv.plot(kind='scatter', x='Newspaper', y='Sales', s=32, alpha=.8)
plt.gca().spines[['top', 'right',]].set_visible(False)
```



```
In [ ]: #Data Cleaning
adv.isnull().sum()
```

```
Out[ ]:
```

	0
TV	0
Radio	0
Newspaper	0
Sales	0

dtype: int64

```
In [ ]: adv.dtypes
```

```
Out[ ]:
```

	0
TV	float64
Radio	float64
Newspaper	float64
Sales	float64

dtype: object

```
In [ ]: for i in adv.columns:
        print(i,':','\n',adv[i].unique(),'\n')
```

TV :

```
[230.1 44.5 17.2 151.5 180.8 8.7 57.5 120.2 8.6 199.8 66.1 214.7
23.8 97.5 204.1 195.4 67.8 281.4 69.2 147.3 218.4 237.4 13.2 228.3
62.3 262.9 142.9 240.1 248.8 70.6 292.9 112.9 97.2 265.6 95.7 290.7
266.9 74.7 43.1 228. 202.5 177. 293.6 206.9 25.1 175.1 89.7 239.9
227.2 66.9 100.4 216.4 182.6 262.7 198.9 7.3 136.2 210.8 210.7 53.5
261.3 239.3 102.7 131.1 69. 31.5 139.3 216.8 199.1 109.8 26.8 129.4
213.4 16.9 27.5 120.5 5.4 116. 76.4 239.8 75.3 68.4 213.5 193.2
76.3 110.7 88.3 134.3 28.6 217.7 250.9 107.4 163.3 197.6 184.9 289.7
135.2 222.4 296.4 280.2 187.9 238.2 137.9 25. 90.4 13.1 255.4 225.8
241.7 175.7 209.6 78.2 75.1 139.2 125.7 19.4 141.3 18.8 224. 123.1
229.5 87.2 7.8 80.2 220.3 59.6 0.7 265.2 8.4 219.8 36.9 48.3
25.6 273.7 43. 73.4 193.7 220.5 104.6 96.2 140.3 243.2 38. 44.7
280.7 121. 171.3 187.8 4.1 93.9 149.8 11.7 131.7 172.5 85.7 188.4
163.5 117.2 234.5 17.9 206.8 215.4 284.3 50. 164.5 19.6 168.4 276.9
248.4 170.2 276.7 165.6 156.6 218.5 56.2 287.6 253.8 205. 139.5 191.1
286. 18.7 39.5 75.5 166.8 149.7 38.2 94.2 283.6 232.1]
```

Radio :

```
[37.8 39.3 45.9 41.3 10.8 48.9 32.8 19.6 2.1 2.6 5.8 24. 35.1 7.6
32.9 47.7 36.6 39.6 20.5 23.9 27.7 5.1 15.9 16.9 12.6 3.5 29.3 16.7
27.1 16. 28.3 17.4 1.5 20. 1.4 4.1 43.8 49.4 26.7 37.7 22.3 33.4
8.4 25.7 22.5 9.9 41.5 15.8 11.7 3.1 9.6 41.7 46.2 28.8 28.1 19.2
49.6 29.5 2. 42.7 15.5 29.6 42.8 9.3 24.6 14.5 27.5 43.9 30.6 14.3
33. 5.7 43.7 1.6 28.5 29.9 7.7 20.3 44.5 43. 18.4 40.6 25.5 47.8
4.9 33.5 36.5 14. 31.6 21. 42.3 4.3 36.3 10.1 17.2 34.3 46.4 11.
0.3 0.4 26.9 8.2 38. 15.4 20.6 46.8 35. 0.8 36.9 26.8 21.7 2.4
34.6 32.3 11.8 38.9 0. 49. 12. 2.9 27.2 38.6 47. 39. 28.9 25.9
17. 35.4 33.2 14.8 1.9 7.3 40.3 25.8 13.9 23.3 39.7 21.1 11.6 43.5
1.3 18.1 35.8 36.8 14.7 3.4 37.6 5.2 23.6 10.6 20.9 20.1 7.1 30.2
7.8 2.3 10. 5.4 21.3 45.1 28.7 12.1 41.1 42. 35.6 3.7 8.6]
```

Newspaper :

```
[ 69.2 45.1 69.3 58.5 58.4 75. 23.5 11.6 1. 21.2 24.2 4.
65.9 7.2 46. 52.9 114. 55.8 18.3 19.1 53.4 49.6 26.2 19.5
12.6 22.9 40.8 43.2 38.6 30. 0.3 7.4 8.5 5. 45.7 35.1
32. 31.6 38.7 1.8 26.4 43.3 31.5 35.7 18.5 49.9 36.8 34.6
3.6 39.6 58.7 15.9 60. 41.4 16.6 37.7 9.3 21.4 54.7 27.3
8.4 28.9 0.9 2.2 10.2 11. 27.2 31.7 19.3 31.3 13.1 89.4
20.7 14.2 9.4 23.1 22.3 36.9 32.5 35.6 33.8 65.7 16. 63.2
73.4 51.4 33. 59. 72.3 10.9 5.9 22. 51.2 45.9 49.8 100.9
17.9 5.3 29.7 23.2 25.6 5.5 56.5 2.4 10.7 34.5 52.7 14.8
79.2 46.2 50.4 15.6 12.4 74.2 25.9 50.6 9.2 3.2 43.1 8.7
43. 2.1 65.6 59.7 20.5 1.7 12.9 75.6 37.9 34.4 38.9 9.
44.3 11.9 20.6 37. 48.7 9.5 5.7 50.5 24.3 45.2 30.7 49.3
5.4 84.8 21.6 19.4 57.6 6.4 18.4 47.4 17. 12.8 41.8 20.3
35.2 23.7 17.6 8.3 27.4 71.8 19.6 26.6 18.2 3.7 23.4 5.8
6. 13.8 8.1 66.2]
```

Sales :

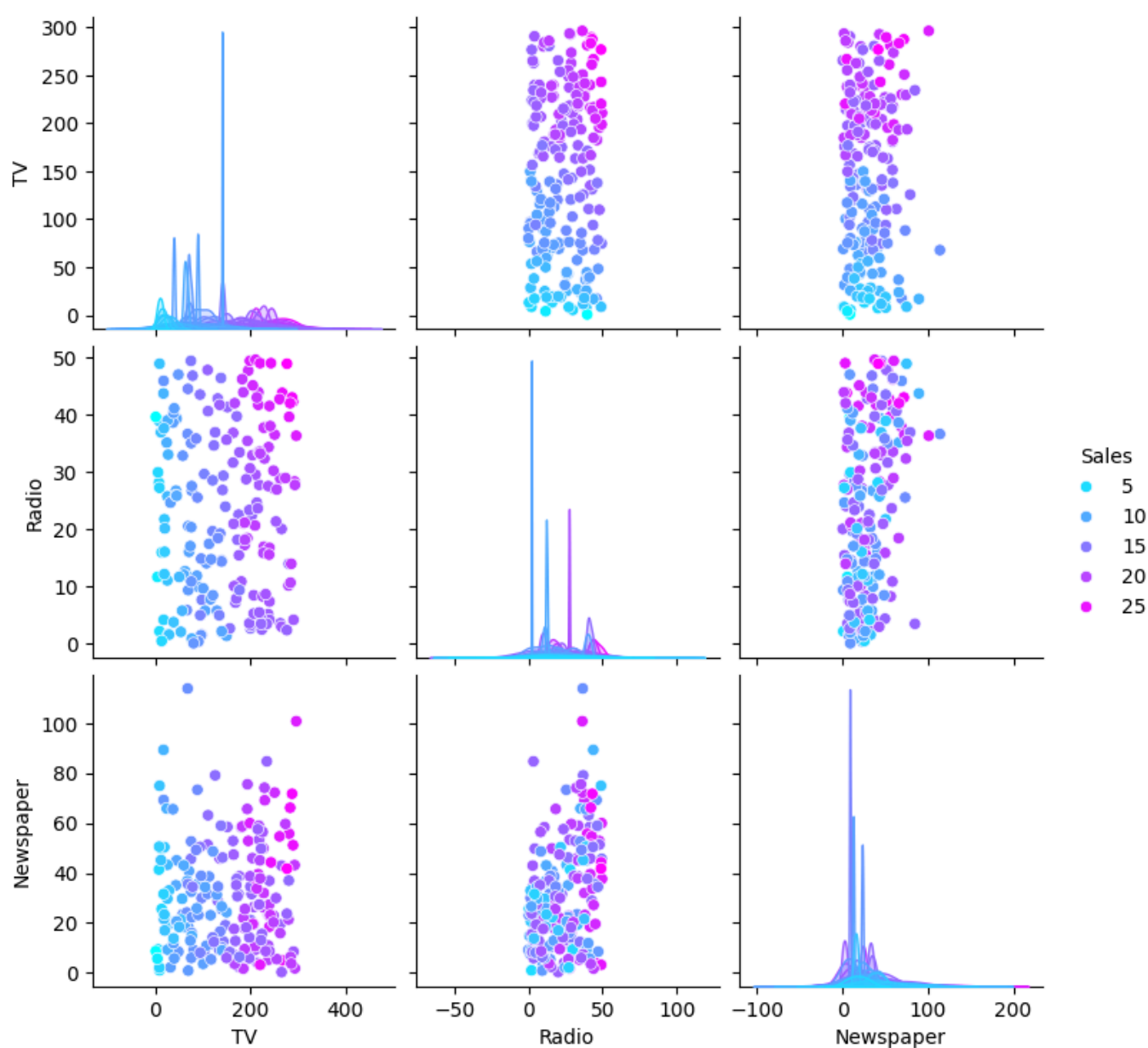
```
[22.1 10.4 12. 16.5 17.9 7.2 11.8 13.2 4.8 15.6 12.6 17.4 9.2 13.7
19. 22.4 12.5 24.4 11.3 14.6 18. 17.5 5.6 20.5 9.7 17. 15. 20.9
18.9 10.5 21.4 11.9 17.8 25.4 14.7 10.1 21.5 16.6 17.1 20.7 8.5 16.1
10.6 23.2 19.8 16.4 10.7 22.6 21.2 20.2 23.7 5.5 23.8 18.4 8.1 24.2
14. 16. 11. 13.4 22.3 18.3 12.4 8.8 8.7 6.9 14.2 5.3 17.3 13.6
21.7 12.9 16.7 7.3 19.4 22.2 11.5 16.9 17.2 19.7 21.8 12.2 9.4 15.9
6.6 15.5 7. 15.2 24.7 1.6 17.7 5.7 19.6 10.8 11.6 9.5 20.8 9.6
10.9 19.2 20.1 12.3 10.3 18.2 20.6 3.2 15.3 13.3 19.9 8. 20. 8.4
7.6 27. 16.8 17.6 26.2 6.7 5.9 14.8 25.5]
```

In []: `adv.describe()`

Out[]:

	TV	Radio	Newspaper	Sales
count	200.000000	200.000000	200.000000	200.000000
mean	147.042500	23.264000	30.554000	15.130500
std	85.854236	14.846809	21.778621	5.283892
min	0.700000	0.000000	0.300000	1.600000
25%	74.375000	9.975000	12.750000	11.000000
50%	149.750000	22.900000	25.750000	16.000000
75%	218.825000	36.525000	45.100000	19.050000
max	296.400000	49.600000	114.000000	27.000000

```
In [ ]: #Data Visualization
sns.pairplot(adv,hue='Sales',palette='cool')
plt.show()
```



```
In [ ]: #Encoding
```

```
In [ ]: #ip/op creation
ip = adv.drop("Sales",axis=1)
op = adv.Sales
```

```
In [ ]: ip.head()
```

Out[]:

	TV	Radio	Newspaper
0	230.1	37.8	69.2
1	44.5	39.3	45.1
2	17.2	45.9	69.3
3	151.5	41.3	58.5
4	180.8	10.8	58.4

```
In [ ]: op.head()
```

Out[]:

	Sales
0	22.1
1	10.4
2	12.0
3	16.5
4	17.9

dtype: float64

```
In [ ]: #Train Test Split
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(ip,op,test_size=0.2)
```

```
In [ ]: x_train
```

Out[]:

	TV	Radio	Newspaper
14	204.1	32.9	46.0
157	149.8	1.3	24.3
66	31.5	24.6	2.2
185	205.0	45.1	19.6
51	100.4	9.6	3.6
...	...	...	...
48	227.2	15.8	49.9
90	134.3	4.9	9.3
164	117.2	14.7	5.4
57	136.2	19.2	16.6
28	248.8	27.1	22.9

160 rows × 3 columns

```
In [ ]: print(x_train.shape)
print(y_train.shape)
```

```
print(x_test.shape)
print(y_test.shape)
```

```
(160, 3)
(160,)
(40, 3)
(40,)
```

```
In [ ]: #Standard Scaler
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()

x_train = sc.fit_transform(x_train)
x_test = sc.fit_transform(x_test)
```

```
In [ ]: #Model Training
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam
```

```
In [ ]: model = Sequential()
#Input & First Hidden Layer
model.add(Dense(30,input_dim=3,activation='linear'))
#Hidden Layers
model.add(Dense(25,activation='linear'))
model.add(Dense(15,activation='linear'))
#Output Layer
model.add(Dense(1,activation='linear'))
model.compile(Adam(learning_rate=0.01),loss='mse')
```

```
In [ ]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 30)	120
dense_5 (Dense)	(None, 25)	775
dense_6 (Dense)	(None, 15)	390
dense_7 (Dense)	(None, 1)	16

Total params: 1,301 (5.08 KB)

Trainable params: 1,301 (5.08 KB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: model.fit(x_train,y_train,validation_data=(x_test,y_test),epochs=25)
```

```

Epoch 1/25
5/5 ————— 2s 54ms/step - loss: 259.1104 - val_loss: 190.1341
Epoch 2/25
5/5 ————— 0s 16ms/step - loss: 198.6769 - val_loss: 111.2210
Epoch 3/25
5/5 ————— 0s 19ms/step - loss: 111.4179 - val_loss: 31.4138
Epoch 4/25
5/5 ————— 0s 16ms/step - loss: 28.4405 - val_loss: 16.2135
Epoch 5/25
5/5 ————— 0s 17ms/step - loss: 16.0991 - val_loss: 23.0671
Epoch 6/25
5/5 ————— 0s 27ms/step - loss: 15.6794 - val_loss: 7.9197
Epoch 7/25
5/5 ————— 0s 19ms/step - loss: 5.3428 - val_loss: 5.9972
Epoch 8/25
5/5 ————— 0s 16ms/step - loss: 5.0196 - val_loss: 4.1271
Epoch 9/25
5/5 ————— 0s 16ms/step - loss: 3.9059 - val_loss: 2.5384
Epoch 10/25
5/5 ————— 0s 16ms/step - loss: 4.5130 - val_loss: 3.2758
Epoch 11/25
5/5 ————— 0s 16ms/step - loss: 2.9283 - val_loss: 4.2256
Epoch 12/25
5/5 ————— 0s 17ms/step - loss: 2.7083 - val_loss: 3.4306
Epoch 13/25
5/5 ————— 0s 17ms/step - loss: 3.2080 - val_loss: 4.1015
Epoch 14/25
5/5 ————— 0s 17ms/step - loss: 3.6075 - val_loss: 2.9961
Epoch 15/25
5/5 ————— 0s 29ms/step - loss: 3.1894 - val_loss: 3.2229
Epoch 16/25
5/5 ————— 0s 17ms/step - loss: 2.8833 - val_loss: 3.1380
Epoch 17/25
5/5 ————— 0s 17ms/step - loss: 2.4655 - val_loss: 2.6707
Epoch 18/25
5/5 ————— 0s 17ms/step - loss: 3.1295 - val_loss: 3.6737
Epoch 19/25
5/5 ————— 0s 17ms/step - loss: 2.7168 - val_loss: 2.9519
Epoch 20/25
5/5 ————— 0s 21ms/step - loss: 2.9690 - val_loss: 3.2217
Epoch 21/25
5/5 ————— 0s 17ms/step - loss: 2.6022 - val_loss: 3.5510
Epoch 22/25
5/5 ————— 0s 16ms/step - loss: 2.8017 - val_loss: 2.7997
Epoch 23/25
5/5 ————— 0s 18ms/step - loss: 2.8781 - val_loss: 3.3217
Epoch 24/25
5/5 ————— 0s 16ms/step - loss: 2.4881 - val_loss: 3.1666
Epoch 25/25
5/5 ————— 0s 17ms/step - loss: 2.5607 - val_loss: 3.1097

```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7bbf41246850>
```

```
In [ ]: pred = model.predict(x_test)
        pred
```

```
2/2 ————— 0s 55ms/step
```

```
Out[ ]: array([[14.700313 ],
               [ 6.765051 ],
               [19.629183 ],
               [13.351385 ],
               [26.0659   ],
               [ 8.521109 ],
               [ 9.818052 ],
               [18.452015 ],
               [10.90967  ],
               [11.746205 ],
               [15.673532 ],
               [19.946362 ],
               [22.51372  ],
               [13.030692 ],
               [22.146315 ],
               [13.16293  ],
               [14.6309185],
               [ 6.016799 ],
               [ 8.623535 ],
               [14.638393 ],
               [16.162622 ],
               [13.154075 ],
               [13.453216 ],
               [19.956144 ],
               [21.31172  ],
               [13.415699 ],
               [17.830627 ],
               [25.044312 ],
               [16.912333 ],
               [18.181358 ],
               [10.719237 ],
               [17.822113 ],
               [20.476545 ],
               [17.715584 ],
               [12.847677 ],
               [ 9.023304 ],
               [14.692264 ],
               [20.5991   ],
               [10.129969 ],
               [11.65283  ]], dtype=float32)
```

```
In [ ]: #Accuracy
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(pred,y_test)
r2 = r2_score(pred,y_test)

print("MSE : ",mse)
print("R2 Score : ",r2)
```

```
MSE :  3.1096764532841226
R2 Score :  0.8687533097833053
```

```
In [ ]:
```

For Classification Models

```
In [ ]: #Importing Packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: #Reading Dataset
iris = pd.read_csv(r'/content/iris.data',header=None)
iris
```

```
Out[ ]:
```

	0	1	2	3	4
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

```
In [ ]: iris.columns = ['SL','SW','PL','PW','Flower']
iris.head()
```

```
Out[ ]:
```

	SL	SW	PL	PW	Flower
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [ ]: #Data Cleaning
iris.isnull().sum()
```

```
Out[ ]:
```

	0
SL	0
SW	0
PL	0
PW	0
Flower	0

dtype: int64

```
In [ ]: iris.dtypes
```

Out []: 0

SL float64

SW float64

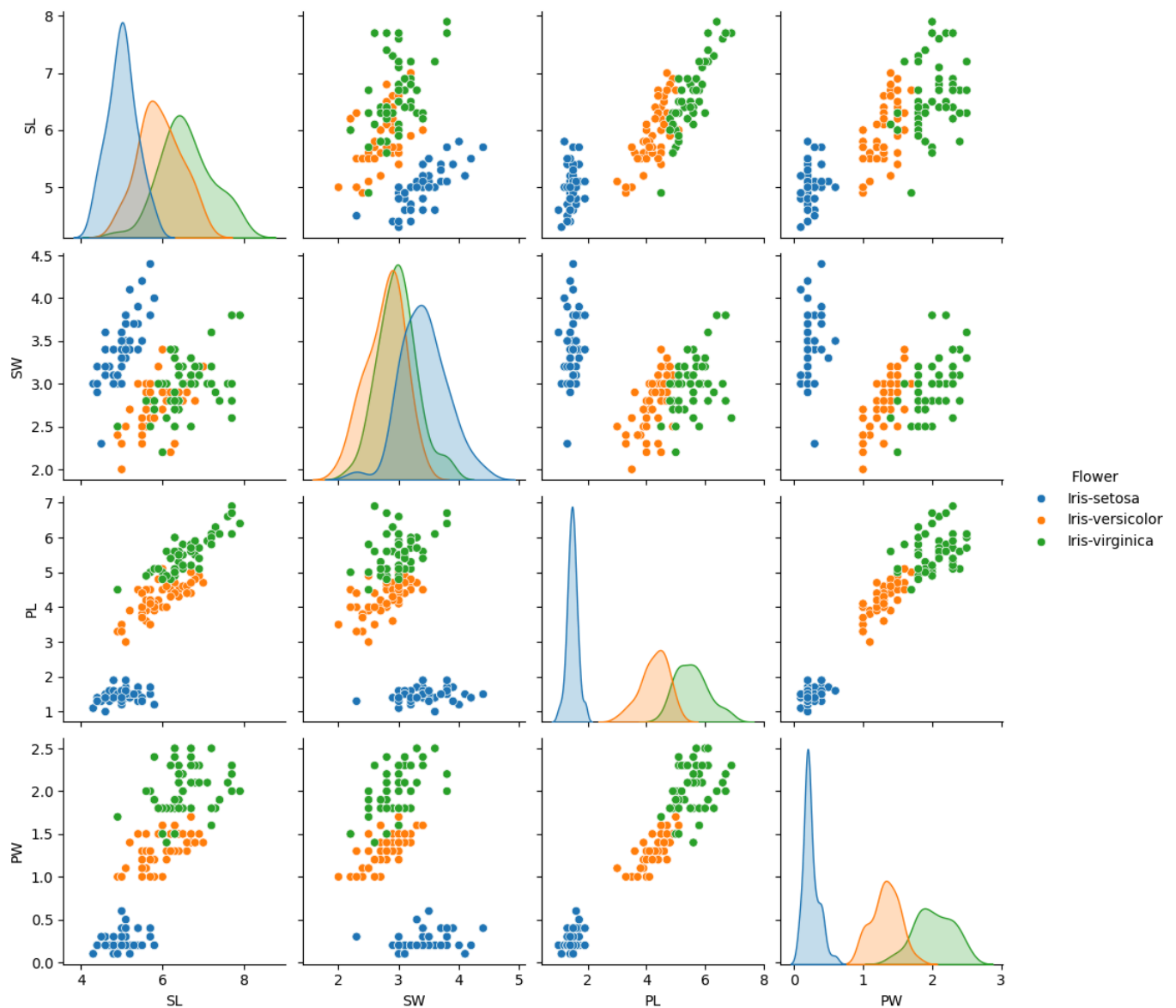
PL float64

PW float64

Flower object

dtype: object

```
In [ ]: #Data Visualization
sns.pairplot(iris,hue='Flower')
plt.show()
```



```
In [ ]: #Encoding
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

```
In [ ]: iris.Flower = le.fit_transform(iris.Flower)
iris.Flower.unique()
```

Out []: array([0, 1, 2])

```
In [ ]: le.inverse_transform([0,1,2])
```

```
Out[ ]: array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)
```

```
In [ ]: iris
```

```
Out[ ]:
```

	SL	SW	PL	PW	Flower
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

```
In [ ]: #ip/op Creation
ip = iris.drop("Flower",axis=1)
op = iris.Flower
```

```
In [ ]: #to_categorical
from keras.utils import to_categorical
op = to_categorical(op,3)
```

```
In [ ]: op
```

[illegible]

[illegible]

```
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.]]
```

```
In [ ]: #Train Test Split
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(ip,op,test_size=0.2)
```

```
In [ ]: #Standard Scaler
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()

x_train = sc.fit_transform(x_train)
x_test = sc.fit_transform(x_test)
```

```
In [ ]: #Model Training
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam
```

```
In [ ]: model = Sequential()
#input and hidden layer
model.add(Dense(20,input_dim=4,activation='relu'))
#hidden layer
model.add(Dense(30,activation='relu'))
model.add(Dense(10,activation='relu'))
#output layer
model.add(Dense(3,activation='softmax'))
model.compile(Adam(learning_rate=0.01),loss='categorical_crossentropy',metrics=['accuracy'])
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
In [ ]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 20)	100
dense_1 (Dense)	(None, 30)	630
dense_2 (Dense)	(None, 10)	310
dense_3 (Dense)	(None, 3)	33



Total params: 1,073 (4.19 KB)

Trainable params: 1,073 (4.19 KB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: model.fit(x_train,y_train,validation_data=(x_test,y_test),epochs=20)
```

```

Epoch 1/20
4/4  2s 93ms/step - accuracy: 0.3444 - loss: 1.2017 - val_accuracy: 0.8000
- val_loss: 0.9058
Epoch 2/20
4/4  0s 24ms/step - accuracy: 0.7983 - loss: 0.8817 - val_accuracy: 0.8000
- val_loss: 0.6748
Epoch 3/20
4/4  0s 25ms/step - accuracy: 0.8125 - loss: 0.6862 - val_accuracy: 0.8000
- val_loss: 0.4650
Epoch 4/20
4/4  0s 23ms/step - accuracy: 0.8623 - loss: 0.4577 - val_accuracy: 0.8000
- val_loss: 0.3622
Epoch 5/20
4/4  0s 25ms/step - accuracy: 0.8733 - loss: 0.3355 - val_accuracy: 0.8000
- val_loss: 0.3581
Epoch 6/20
4/4  0s 24ms/step - accuracy: 0.8677 - loss: 0.2877 - val_accuracy: 0.8667
- val_loss: 0.2847
Epoch 7/20
4/4  0s 26ms/step - accuracy: 0.9248 - loss: 0.2189 - val_accuracy: 0.8000
- val_loss: 0.3028
Epoch 8/20
4/4  0s 25ms/step - accuracy: 0.8777 - loss: 0.2236 - val_accuracy: 0.8333
- val_loss: 0.2690
Epoch 9/20
4/4  0s 23ms/step - accuracy: 0.9708 - loss: 0.1277 - val_accuracy: 0.9000
- val_loss: 0.2202
Epoch 10/20
4/4  0s 23ms/step - accuracy: 0.9585 - loss: 0.1160 - val_accuracy: 0.9000
- val_loss: 0.2535
Epoch 11/20
4/4  0s 23ms/step - accuracy: 0.9681 - loss: 0.0957 - val_accuracy: 0.9000
- val_loss: 0.2335
Epoch 12/20
4/4  0s 23ms/step - accuracy: 0.9744 - loss: 0.0685 - val_accuracy: 0.9000
- val_loss: 0.2448
Epoch 13/20
4/4  0s 24ms/step - accuracy: 0.9796 - loss: 0.0602 - val_accuracy: 0.9333
- val_loss: 0.2402
Epoch 14/20
4/4  0s 39ms/step - accuracy: 0.9806 - loss: 0.0539 - val_accuracy: 0.9000
- val_loss: 0.2806
Epoch 15/20
4/4  0s 44ms/step - accuracy: 0.9798 - loss: 0.0375 - val_accuracy: 0.9667
- val_loss: 0.1887
Epoch 16/20
4/4  0s 44ms/step - accuracy: 0.9775 - loss: 0.0550 - val_accuracy: 0.9000
- val_loss: 0.3295
Epoch 17/20
4/4  0s 35ms/step - accuracy: 0.9852 - loss: 0.0608 - val_accuracy: 0.9000
- val_loss: 0.2886
Epoch 18/20
4/4  0s 33ms/step - accuracy: 0.9767 - loss: 0.0559 - val_accuracy: 0.9333
- val_loss: 0.2083
Epoch 19/20
4/4  0s 33ms/step - accuracy: 0.9619 - loss: 0.0596 - val_accuracy: 0.9000
- val_loss: 0.2713
Epoch 20/20
4/4  0s 35ms/step - accuracy: 0.9852 - loss: 0.0477 - val_accuracy: 0.9333
- val_loss: 0.2390

```

```
Out[ ]: <keras.src.callbacks.history.History at 0x789db9dfa910>
```

```
In [ ]: pred = model.predict(x_test)
pred
```

```
Out[ ]: array([[3.0784086e-08, 3.5990542e-04, 9.9964011e-01],
               [9.9998897e-01, 1.0926008e-05, 3.8580941e-17],
               [3.0589842e-06, 4.2145262e-03, 9.9578232e-01],
               [9.9999028e-01, 9.6357826e-06, 1.3902714e-17],
               [9.8363495e-10, 7.5511947e-05, 9.9992448e-01],
               [6.2352614e-05, 9.4157100e-01, 5.8366567e-02],
               [9.9999720e-01, 2.7388046e-06, 1.0729124e-18],
               [9.9993652e-01, 6.3357627e-05, 9.1571557e-16],
               [9.9996477e-01, 3.5179943e-05, 1.7903025e-16],
               [7.5728871e-04, 9.9811894e-01, 1.1236904e-03],
               [1.6267603e-07, 7.6557527e-04, 9.9923432e-01],
               [1.4938168e-04, 3.5146004e-01, 6.4839059e-01],
               [2.7134540e-04, 9.4953537e-01, 5.0193220e-02],
               [9.9995142e-01, 4.8498299e-05, 1.1402087e-15],
               [4.3263686e-08, 4.1995561e-04, 9.9957997e-01],
               [9.9998075e-01, 1.9159716e-05, 1.2277230e-16],
               [9.9999613e-01, 3.7962532e-06, 2.8637336e-18],
               [9.9994010e-01, 5.9869824e-05, 2.8419094e-16],
               [9.9997586e-01, 2.4066527e-05, 6.4571631e-17],
               [1.7748364e-07, 7.9641520e-04, 9.9920338e-01],
               [3.5934729e-06, 3.6987171e-02, 9.6300930e-01],
               [6.2761237e-06, 9.9970520e-01, 2.8857382e-04],
               [2.4680598e-04, 9.9972498e-01, 2.8292923e-05],
               [1.7322405e-04, 6.3339621e-01, 3.6643061e-01],
               [9.9989390e-01, 1.0612717e-04, 6.2874197e-16],
               [5.3516502e-10, 5.7297151e-05, 9.9994266e-01],
               [6.3077394e-05, 9.8289239e-01, 1.7044643e-02],
               [4.5389539e-04, 9.7464120e-01, 2.4904883e-02],
               [3.0581640e-07, 1.0191903e-03, 9.9898046e-01],
               [3.6937886e-06, 9.9997318e-01, 2.3139302e-05]], dtype=float32)
```

```
In [ ]: import numpy as np
```

```
In [ ]: p = np.argmax(pred,axis=1)
p
```

```
Out[ ]: array([2, 0, 2, 0, 2, 1, 0, 0, 0, 1, 2, 2, 1, 0, 2, 0, 0, 0, 0, 2, 2, 1,
               1, 1, 0, 2, 1, 1, 2, 1])
```

```
In [ ]: y_test
```

```
Out[ ]: array([[0., 0., 1.],
               [1., 0., 0.],
               [0., 1., 0.],
               [1., 0., 0.],
               [0., 0., 1.],
               [0., 1., 0.],
               [1., 0., 0.],
               [1., 0., 0.],
               [1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.],
               [0., 1., 0.],
               [0., 1., 0.],
               [1., 0., 0.],
               [0., 0., 1.],
               [1., 0., 0.],
               [1., 0., 0.],
               [1., 0., 0.],
               [1., 0., 0.],
               [0., 0., 1.],
               [0., 0., 1.],
               [0., 1., 0.],
               [0., 1., 0.],
               [0., 1., 0.],
               [1., 0., 0.],
               [0., 0., 1.],
               [0., 1., 0.],
               [0., 1., 0.],
               [0., 0., 1.],
               [0., 1., 0.]])
```

```
In [ ]: y = np.argmax(y_test,axis=1)
        y
```

```
Out[ ]: array([2, 0, 1, 0, 2, 1, 0, 0, 0, 1, 2, 1, 1, 0, 2, 0, 0, 0, 2, 2, 1,
               1, 1, 0, 2, 1, 1, 2, 1])
```

```
In [ ]: from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score

ac = accuracy_score(p,y)
re = recall_score(p,y,average='micro')
pr = precision_score(p,y,average='micro')
f1 = f1_score(p,y,average='micro')

print("Accuracy : ",ac)
print("Recall : ",re)
print("Precision : ",pr)
print("F1 Score : ",f1)
```

```
Accuracy :  0.9333333333333333
Recall :    0.9333333333333333
Precision : 0.9333333333333333
F1 Score :  0.9333333333333333
```

```
In [ ]:
```

```
In [ ]:
```

MNIST Dataset using ANN Model

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
```

```
from keras.datasets import mnist

from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam

import random
np.random.seed(0)
```

```
In [ ]: (x_train,y_train), (x_test,y_test) = mnist.load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 0s 0us/step

```
In [ ]: print(x_train.shape)
        print(y_train.shape)
        print(x_test.shape)
        print(y_test.shape)
```

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

```
In [ ]: x_train
```

```
Out[ ]: array([[0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               ...,
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0]],

               [[0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                ...,
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0]],

               [[0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                ...,
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0]],

               ...,

               [[0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                ...,
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0]],

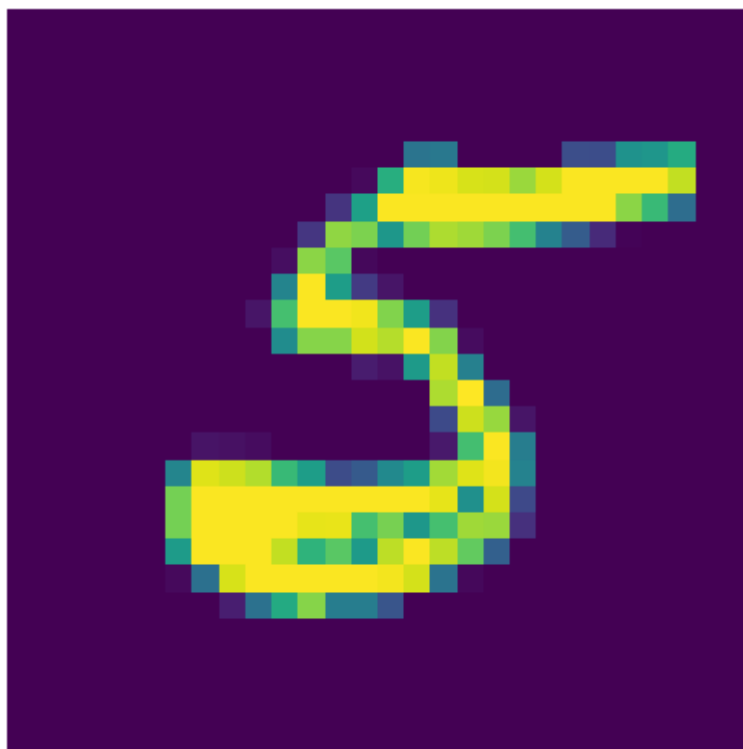
               [[0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                ...,
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0]],

               [[0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                ...,
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0],
                [0, 0, 0, ..., 0, 0, 0]]], dtype=uint8)
```

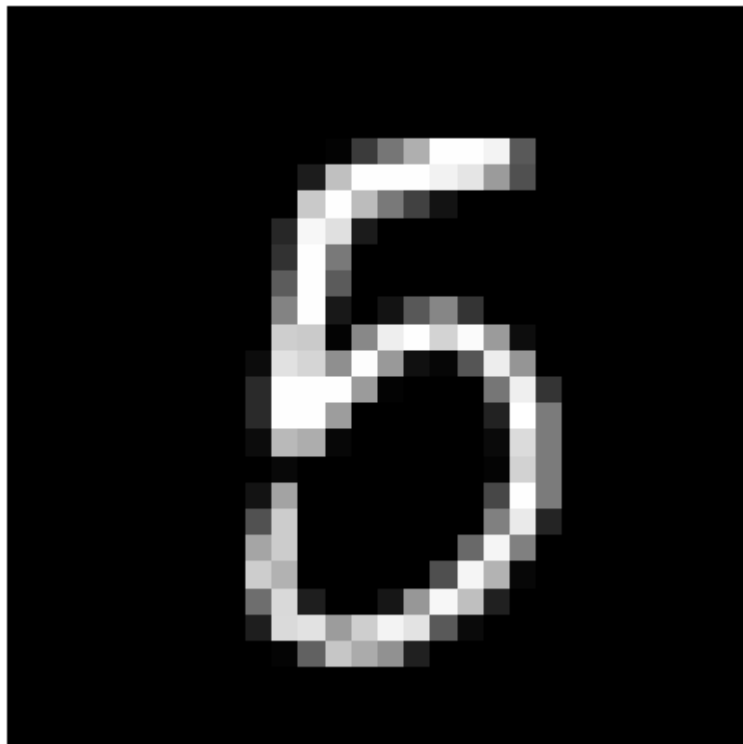
```
In [ ]: set(y_train)
```

```
Out[ ]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
In [ ]: plt.imshow(x_train[y_train==5][100])
plt.axis('off')
plt.show()
```



```
In [ ]: plt.imshow(x_train[y_train==5][5000], cmap='gray')
plt.axis('off')
plt.show()
```

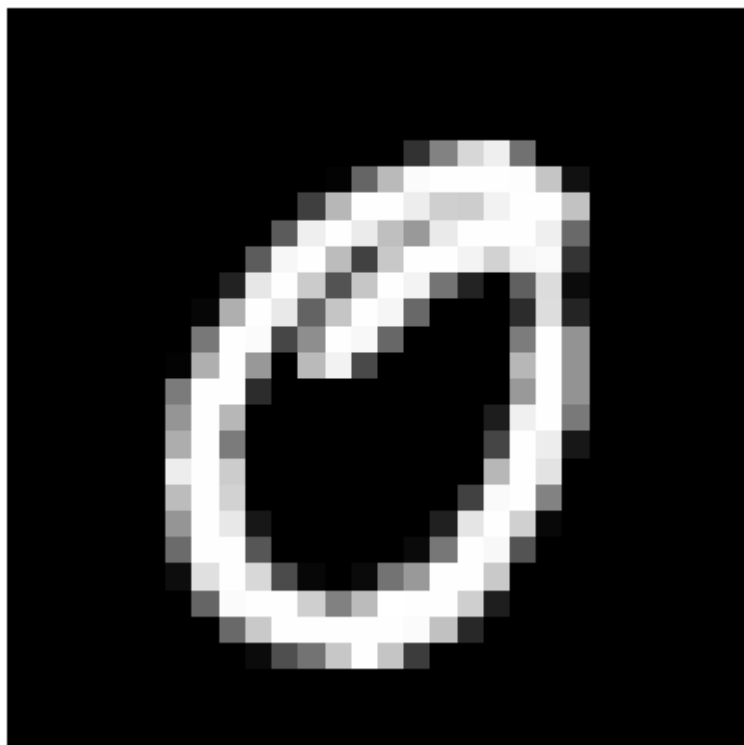


```
In [ ]: for i in range(0,10):
print(i,':',len(x_train[y_train==i]))
```

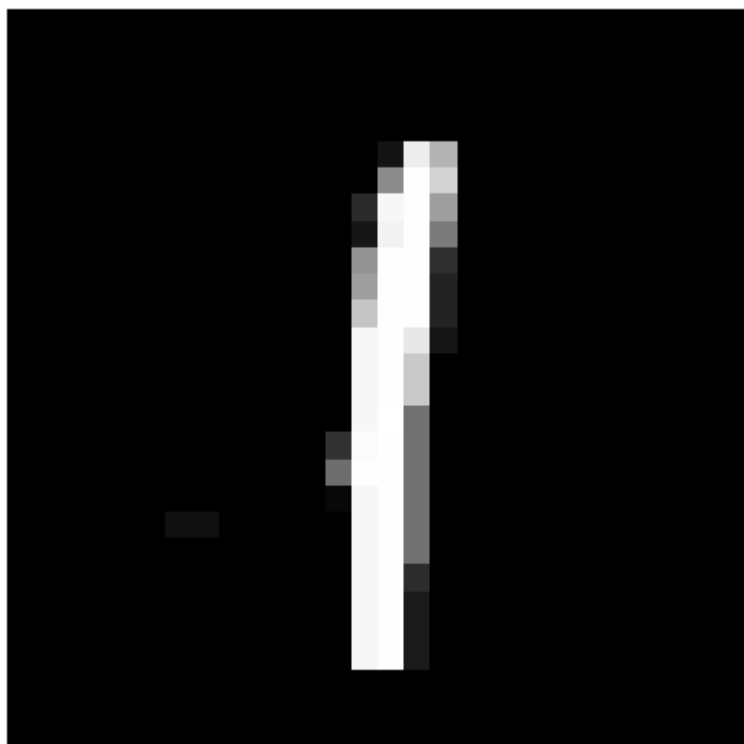
```
0 : 5923
1 : 6742
2 : 5958
3 : 6131
4 : 5842
5 : 5421
6 : 5918
7 : 6265
8 : 5851
9 : 5949
```

```
In [ ]: for i in range(0,10):  
        plt.imshow(x_train[y_train==i][np.random.randint(0,5000)], cmap='gray')  
        plt.title(str(i))  
        plt.axis('off')  
        plt.show()
```

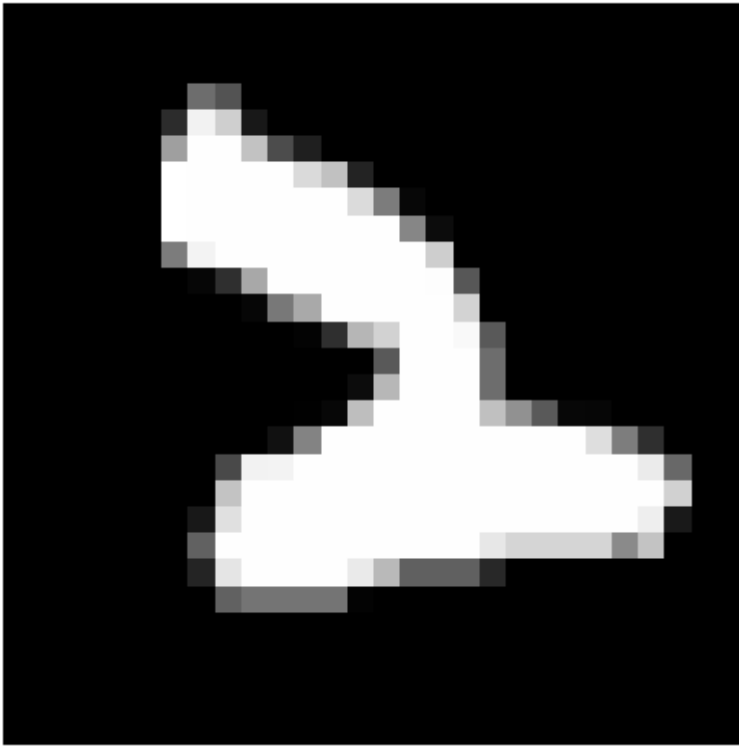
0



1



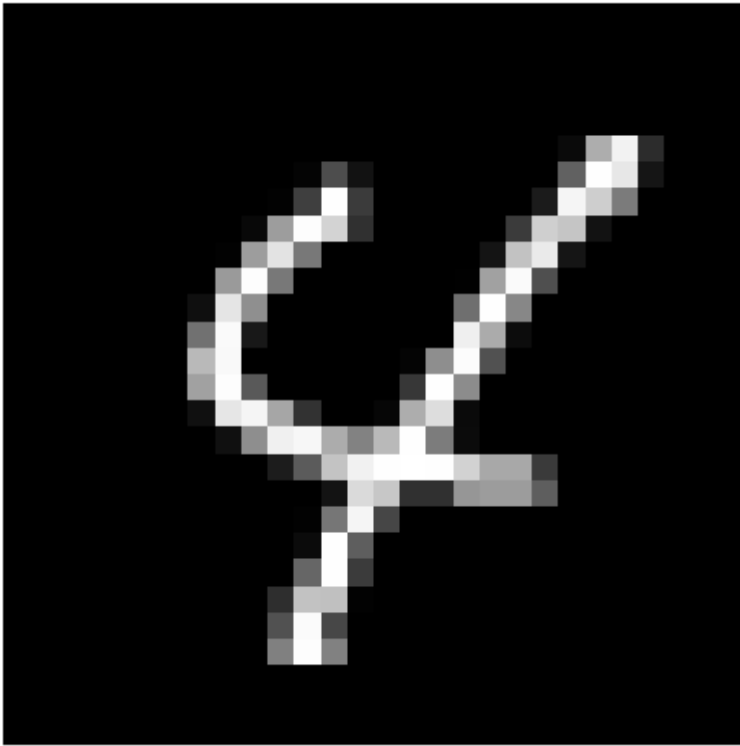
2



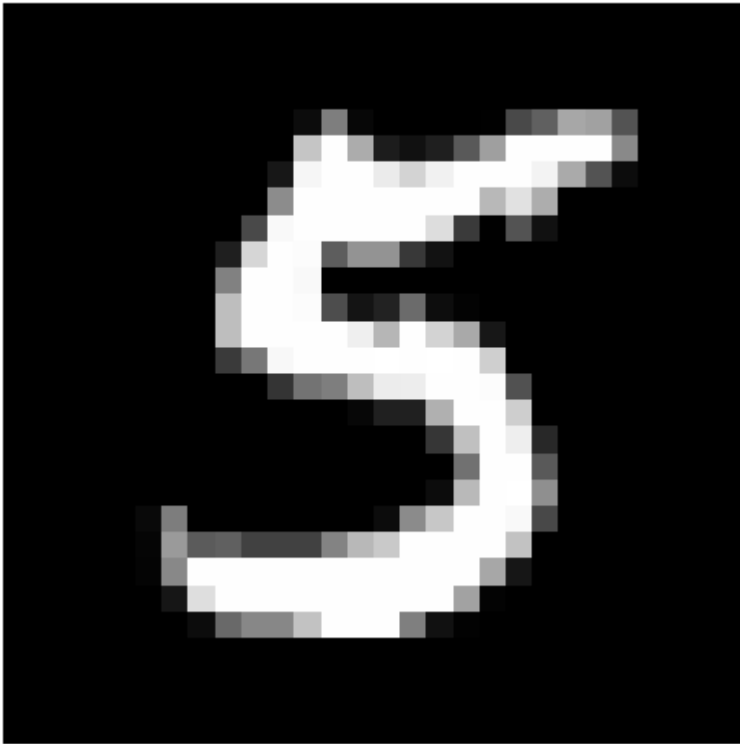
3



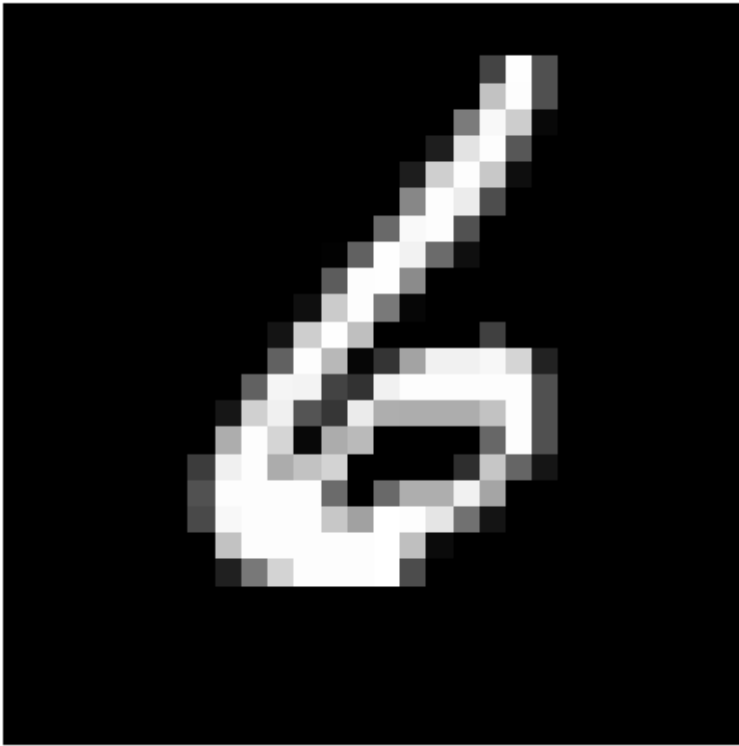
4



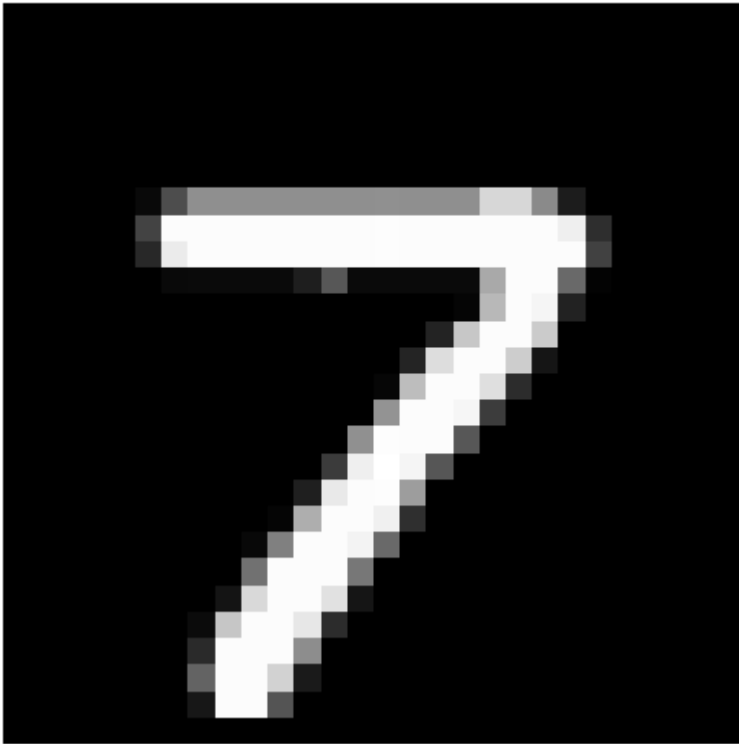
5



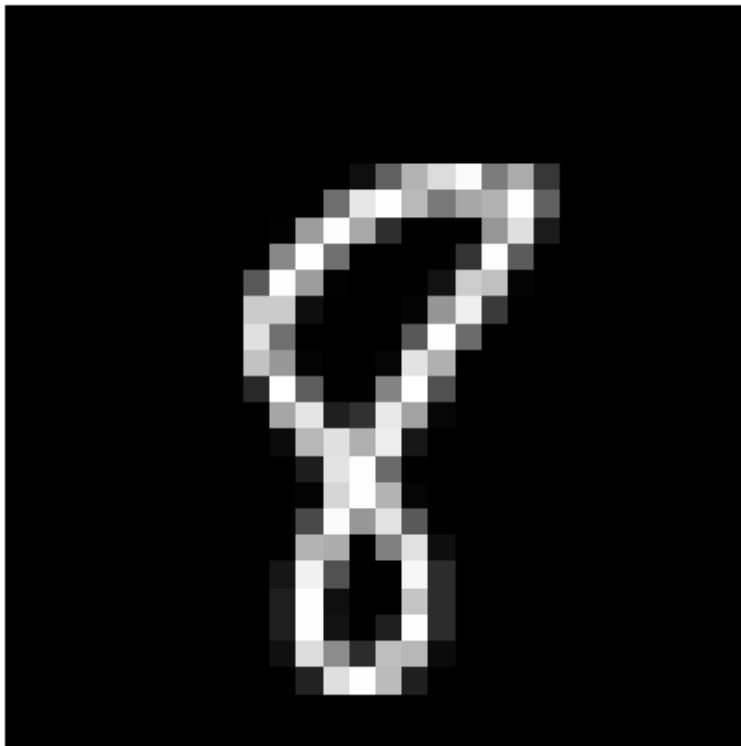
6



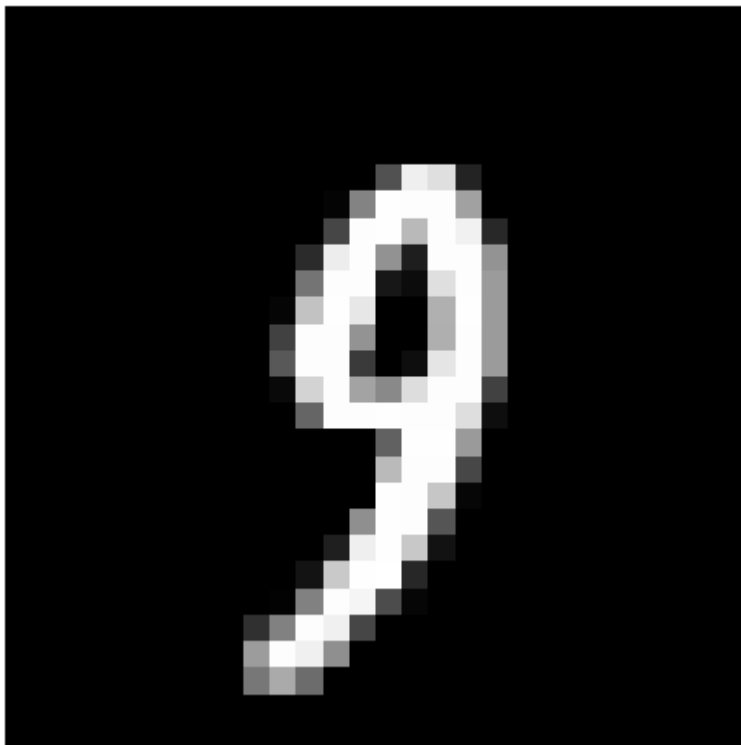
7



8



9



```
In [ ]: #to_categorical  
        from keras.utils import to_categorical
```

```
In [ ]: y_train = to_categorical(y_train,10)  
        y_test = to_categorical(y_test)
```

```
In [ ]: x_train = x_train/255  
        x_test = x_test/255
```

```
In [ ]: x_train
```

```
Out[ ]: array([[0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               ...,
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.]],

               [[0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                ...,
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.]],

               [[0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                ...,
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.]],

               ...,

               [[0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                ...,
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.]],

               [[0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                ...,
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.]],

               [[0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                ...,
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.]])
```

```
In [ ]: y_train
```

```
Out[ ]: array([[0., 0., 0., ..., 0., 0., 0.],
               [1., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               ...,
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 1., 0.]])
```

```
In [ ]: x_train.shape
```

```
Out[ ]: (60000, 28, 28)
```

```
In [ ]: x_train = x_train.reshape(60000,784)
        x_test = x_test.reshape(10000,784)
```

```
In [ ]: x_train.shape
```

```
Out[ ]: (60000, 784)
```

```
In [ ]: #Model Training
        model = Sequential()
        #input and first hidden layer
        model.add(Dense(35,input_dim=784,activation='relu'))
        #hidden layers
        model.add(Dense(15,activation='relu'))
        model.add(Dense(25,activation='relu'))
        #output layer
        model.add(Dense(10,activation='softmax'))
        model.compile(Adam(learning_rate=0.01),loss='categorical_crossentropy',metrics=['accuracy'])
```

```
In [ ]: model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 35)	27,475
dense_9 (Dense)	(None, 15)	540
dense_10 (Dense)	(None, 25)	400
dense_11 (Dense)	(None, 10)	260



Total params: 28,675 (112.01 KB)

Trainable params: 28,675 (112.01 KB)

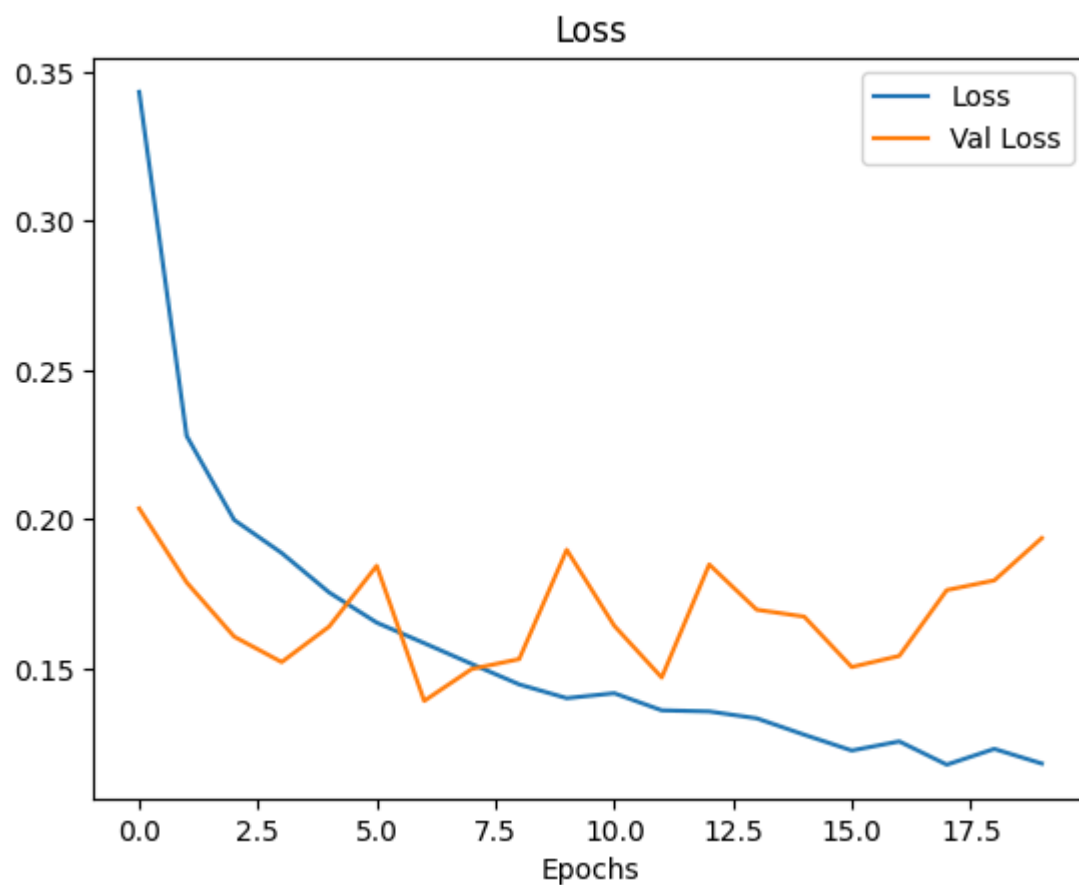
Non-trainable params: 0 (0.00 B)

```
In [ ]: h = model.fit(x_train,y_train,validation_split=0.1,epochs=20)
```

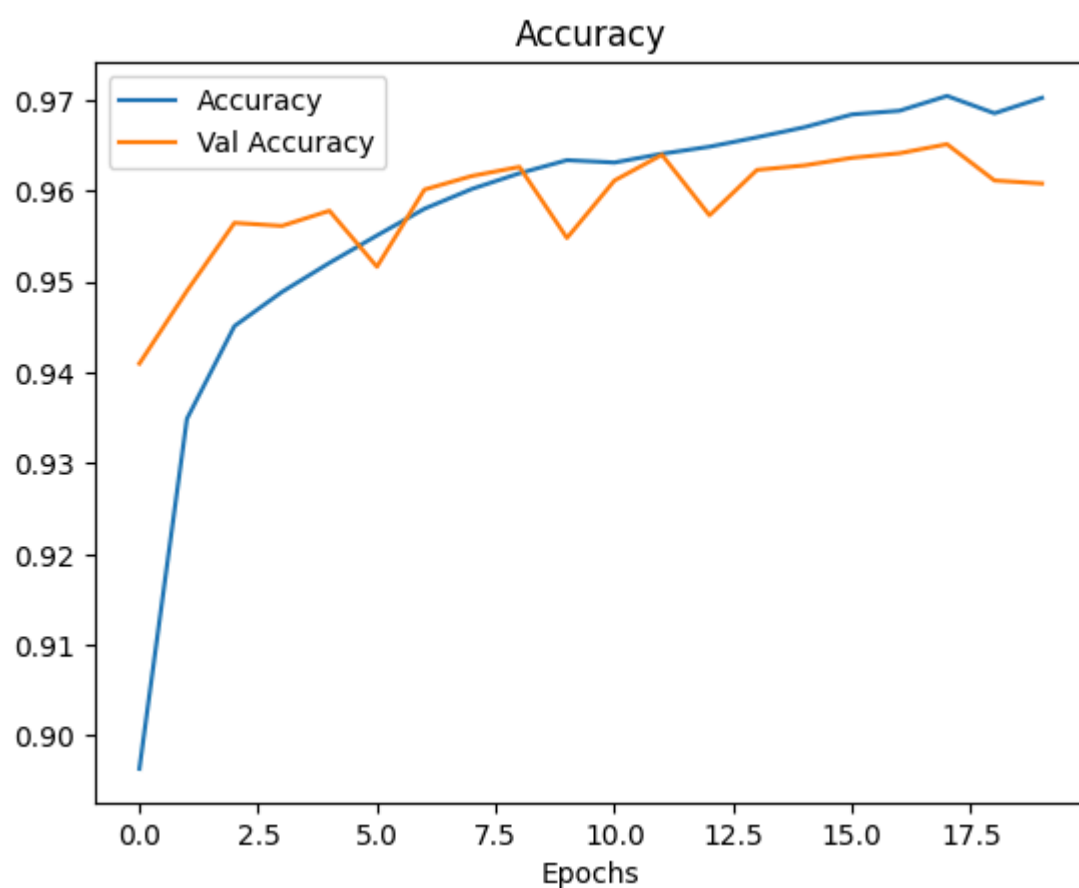
Epoch 1/20
1688/1688 ————— **8s** 3ms/step - accuracy: 0.8378 - loss: 0.5062 - val_accuracy: 0.9410 - val_loss: 0.2036
Epoch 2/20
1688/1688 ————— **9s** 2ms/step - accuracy: 0.9334 - loss: 0.2295 - val_accuracy: 0.9490 - val_loss: 0.1788
Epoch 3/20
1688/1688 ————— **5s** 3ms/step - accuracy: 0.9447 - loss: 0.1987 - val_accuracy: 0.9565 - val_loss: 0.1606
Epoch 4/20
1688/1688 ————— **10s** 3ms/step - accuracy: 0.9498 - loss: 0.1845 - val_accuracy: 0.9562 - val_loss: 0.1521
Epoch 5/20
1688/1688 ————— **5s** 3ms/step - accuracy: 0.9515 - loss: 0.1745 - val_accuracy: 0.9578 - val_loss: 0.1640
Epoch 6/20
1688/1688 ————— **5s** 3ms/step - accuracy: 0.9563 - loss: 0.1585 - val_accuracy: 0.9517 - val_loss: 0.1844
Epoch 7/20
1688/1688 ————— **10s** 6ms/step - accuracy: 0.9583 - loss: 0.1581 - val_accuracy: 0.9602 - val_loss: 0.1391
Epoch 8/20
1688/1688 ————— **7s** 4ms/step - accuracy: 0.9617 - loss: 0.1464 - val_accuracy: 0.9617 - val_loss: 0.1498
Epoch 9/20
1688/1688 ————— **6s** 4ms/step - accuracy: 0.9614 - loss: 0.1455 - val_accuracy: 0.9627 - val_loss: 0.1531
Epoch 10/20
1688/1688 ————— **6s** 4ms/step - accuracy: 0.9636 - loss: 0.1385 - val_accuracy: 0.9548 - val_loss: 0.1897
Epoch 11/20
1688/1688 ————— **5s** 3ms/step - accuracy: 0.9627 - loss: 0.1433 - val_accuracy: 0.9612 - val_loss: 0.1643
Epoch 12/20
1688/1688 ————— **13s** 5ms/step - accuracy: 0.9644 - loss: 0.1345 - val_accuracy: 0.9640 - val_loss: 0.1469
Epoch 13/20
1688/1688 ————— **10s** 5ms/step - accuracy: 0.9658 - loss: 0.1302 - val_accuracy: 0.9573 - val_loss: 0.1848
Epoch 14/20
1688/1688 ————— **6s** 3ms/step - accuracy: 0.9663 - loss: 0.1328 - val_accuracy: 0.9623 - val_loss: 0.1696
Epoch 15/20
1688/1688 ————— **11s** 4ms/step - accuracy: 0.9684 - loss: 0.1266 - val_accuracy: 0.9628 - val_loss: 0.1673
Epoch 16/20
1688/1688 ————— **8s** 5ms/step - accuracy: 0.9684 - loss: 0.1221 - val_accuracy: 0.9637 - val_loss: 0.1504
Epoch 17/20
1688/1688 ————— **10s** 5ms/step - accuracy: 0.9698 - loss: 0.1222 - val_accuracy: 0.9642 - val_loss: 0.1542
Epoch 18/20
1688/1688 ————— **7s** 4ms/step - accuracy: 0.9727 - loss: 0.1100 - val_accuracy: 0.9652 - val_loss: 0.1762
Epoch 19/20
1688/1688 ————— **4s** 3ms/step - accuracy: 0.9674 - loss: 0.1276 - val_accuracy: 0.9612 - val_loss: 0.1795
Epoch 20/20
1688/1688 ————— **5s** 3ms/step - accuracy: 0.9724 - loss: 0.1079 - val_accuracy: 0.9608 - val_loss: 0.1937

```
In [ ]: plt.plot(h.history['loss'])
plt.plot(h.history['val_loss'])
plt.legend(['Loss', 'Val Loss'])
plt.title("Loss")
```

```
plt.xlabel("Epochs")  
plt.show()
```



```
In [ ]: plt.plot(h.history['accuracy'])  
plt.plot(h.history['val_accuracy'])  
plt.legend(['Accuracy', 'Val Accuracy'])  
plt.title("Accuracy")  
plt.xlabel("Epochs")  
plt.show()
```

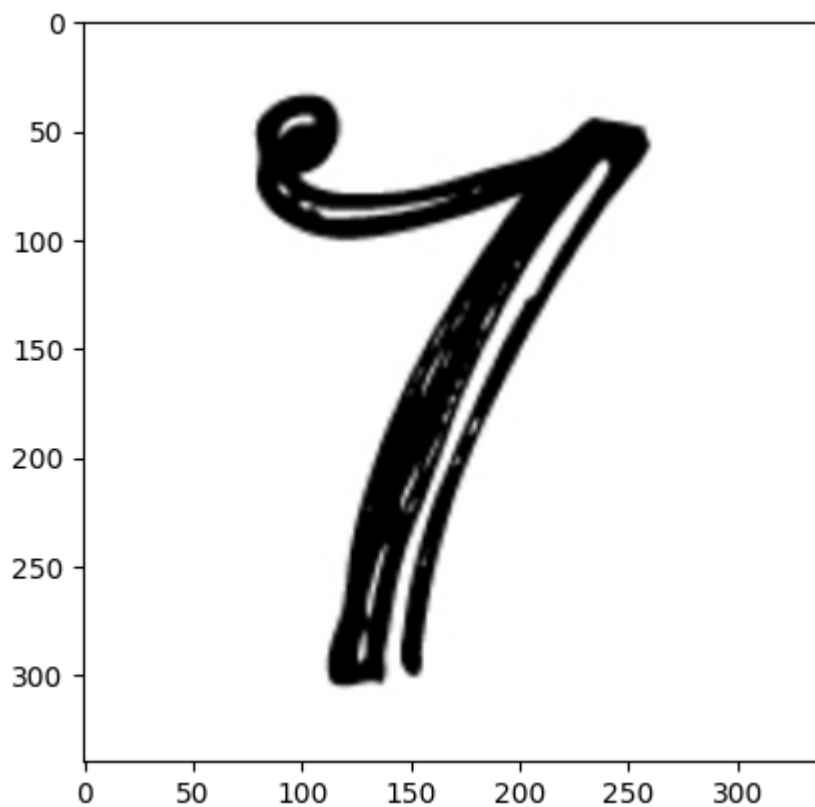


In []:

Method 1 for Testing the Model

```
In [ ]: import cv2
import matplotlib.pyplot as plt
```

```
In [ ]: img = cv2.imread(r'/content/seven12.jpeg')
plt.imshow(img, cmap='gray')
plt.show()
```

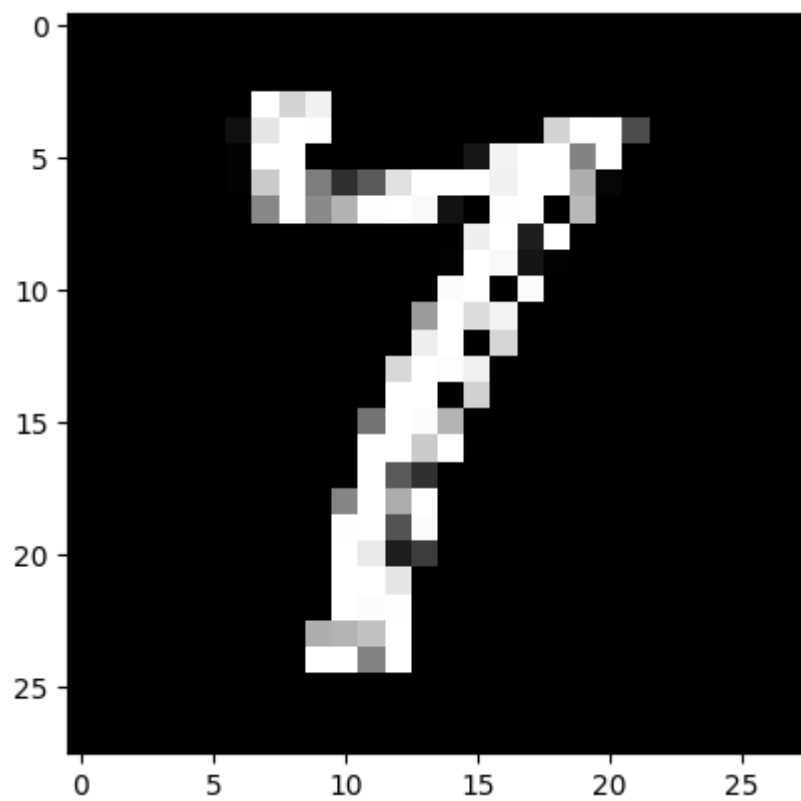


```
In [ ]: img.shape
```

```
Out[ ]: (340, 338, 3)
```

```
In [ ]: img = cv2.resize(img, (28, 28))
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img = cv2.bitwise_not(img)

plt.imshow(img, cmap='gray')
plt.show()
```



```
In [ ]: img = img/255
img = img.reshape(1,784)
```

```
In [ ]: pred = model.predict(img)
pred
```

1/1 ————— 0s 309ms/step

```
Out[ ]: array([[1.6922643e-03, 1.3304092e-01, 5.2530783e-01, 9.3371287e-02,
1.6960442e-02, 1.3968291e-02, 5.6300629e-02, 7.1159981e-02,
8.7917946e-02, 2.8035764e-04]], dtype=float32)
```

```
In [ ]: p = np.argmax(pred,axis=1)
p
```

```
Out[ ]: array([2])
```

```
In [ ]:
```

Method 2 for Testing the Model

```
In [ ]: from google.colab import files
u = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed

in the current browser session. Please rerun this cell to enable.

Saving five.jpeg to five.jpeg

```
In [1]: #print(u)
```

```
In [ ]: d = list(u.keys())[0]
print(d)
```

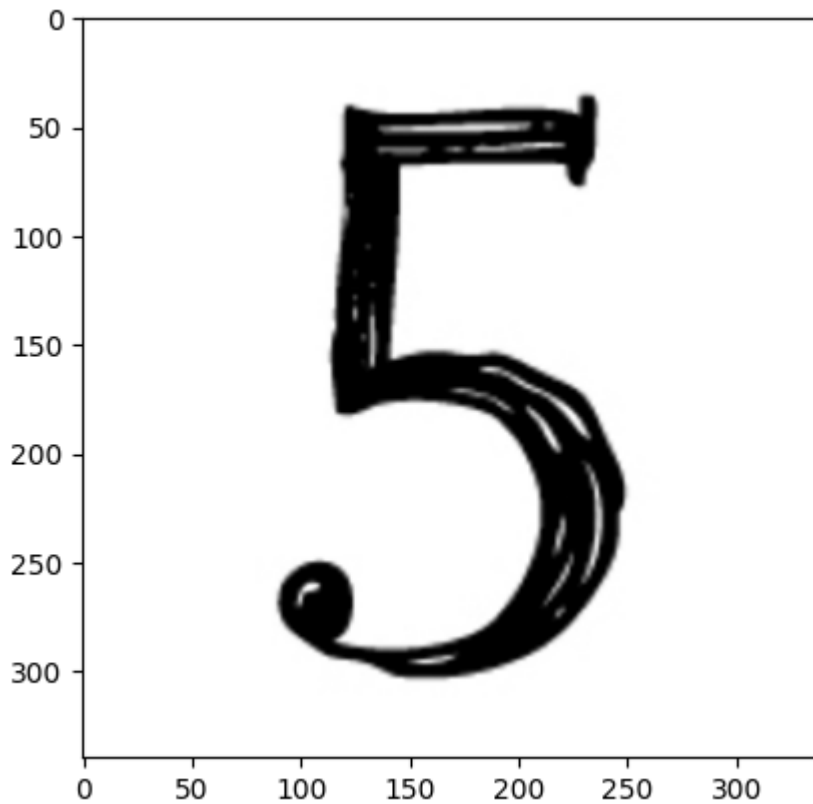
five.jpeg

```
In [ ]: a = np.fromstring(u[d],np.uint8)
print(a)
```

[255 216 255 ... 3 255 217]

```
<ipython-input-72-7d9b404511a3>:1: DeprecationWarning: The binary mode of fromstring is deprecated, as it behaves surprisingly on unicode inputs. Use frombuffer instead
a = np.fromstring(u[d],np.uint8)
```

```
In [ ]: b = cv2.imdecode(a,cv2.IMREAD_COLOR)
plt.imshow(b,cmap='gray')
plt.show()
```

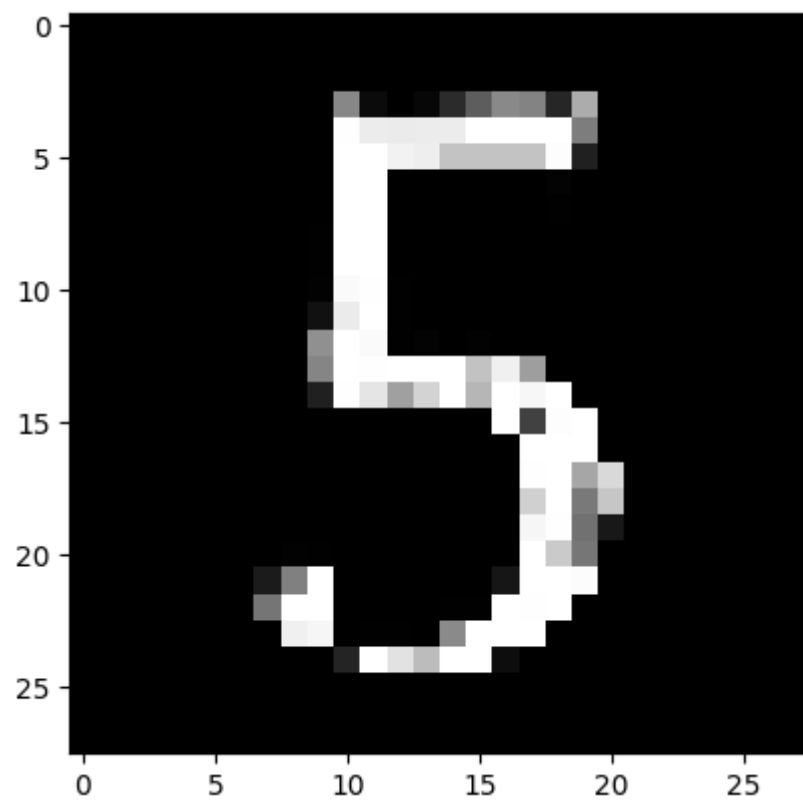


```
In [ ]: b.shape
```

```
Out[ ]: (340, 338, 3)
```

```
In [ ]: b = cv2.resize(b,(28,28))
b = cv2.cvtColor(b,cv2.COLOR_BGR2GRAY)
b = cv2.bitwise_not(b)

plt.imshow(b,cmap='gray')
plt.show()
```



```
In [ ]: b = b/255  
        b = b.reshape(1,784)
```

```
In [ ]: pred = model.predict(b)  
        pred
```

1/1 ————— 0s 77ms/step

```
Out[ ]: array([[2.37062181e-06, 4.85854343e-08, 9.45148770e-07, 2.34677270e-03,  
                8.04675722e-07, 9.84332085e-01, 9.27826681e-04, 2.99609511e-08,  
                3.39758437e-04, 1.20493835e-02]], dtype=float32)
```

```
In [ ]: p = np.argmax(pred,axis=1)  
        p
```

```
Out[ ]: array([5])
```

```
In [ ]:
```